

Lambda

Lambda

Come scrivere funzioni brevi da passare ad altri metodi.

A cosa servono le lambda

Una **lambda** è un modo breve per scrivere una piccola azione.

La usi quando un metodo ti chiede:

"Che cosa devo fare con questo valore?"

Per esempio, una lista può chiederti cosa fare con ogni nome. Tu puoi rispondere con una lambda:

```
nome -> System.out.println(nome)
```

Si legge così:

"prendi un **nome** e stampalo".

La freccia **->** separa due parti:

- a sinistra c'è il valore che arriva
- a destra c'è l'azione da eseguire

La forma più semplice

La forma più comune è questa:

```
parametro -> azione
```

Il **parametro** è il valore che Java passa alla lambda.

L' **azione** è il codice che vuoi eseguire con quel valore.

Esempio:

```
nome -> System.out.println(nome)
```

Qui **nome** non è una variabile creata prima da noi. È il nome che diamo al valore ricevuto dalla lambda.

Usare una lambda con forEach

Vediamo un esempio completo con una lista di nomi.

```
public class EsempioLambda {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Luca");  
        nomi.add("Sara");  
        nomi.add("Mina");  
  
        nomi.forEach(nome -> System.out.println(nome));  
    }  
}
```

Output:

```
Luca  
Sara  
Mina
```

forEach significa "per ciascun elemento".

Quindi questa riga:

```
nomi.forEach(nome -> System.out.println(nome));
```

vuol dire:

"per ogni nome nella lista `nomi` , stampa quel nome".

Java prende un elemento alla volta dalla lista. Ogni elemento viene chiamato `nome` dentro la lambda.

Confronto con il for-each

La stessa cosa si può scrivere anche con un ciclo `for-each` .

```
for (String nome : nomi) {  
    System.out.println(nome);  
}
```

Con una lambda diventa:

```
nomi.forEach(nome -> System.out.println(nome));
```

I due esempi fanno la stessa cosa.

All'inizio il ciclo `for-each` è spesso più facile da leggere. La lambda diventa utile quando usi metodi che vogliono ricevere una piccola azione, come `forEach` , `sort` , `filter` o `map` .

Lambda con più righe

Se l'azione è molto breve, puoi scriverla dopo la freccia.

Se invece servono più righe, usa le parentesi graffe.

```
nomi.forEach(nome -> {  
    String maiuscolo = nome.toUpperCase();  
    System.out.println(maiuscolo);  
});
```

Output:

```
LUCA  
SARA  
MINA
```

Qui succedono due cose per ogni nome:

1. Java crea una versione in maiuscolo.
2. Poi stampa il risultato.

Attenzione: se usi le parentesi graffe, il codice dentro la lambda deve essere scritto come un normale blocco Java.

Lambda che restituisce un valore

Una lambda può anche produrre un risultato.

Per esempio:

```
numero -> numero * 2
```

Si legge:

"prendi `numero` e restituisci `numero * 2`".

In una lambda breve non devi scrivere `return`.

```
Operazione doppio = numero -> numero * 2;
```

Questa lambda prende un numero e produce il suo doppio.

Interfacce funzionali

Una lambda non vive da sola. Java deve sapere che tipo di azione rappresenta.

Per questo una lambda si usa quando Java si aspetta un'interfaccia con **un solo metodo astratto**.

Questa interfaccia si chiama **interfaccia funzionale**.

Esempio:

```
public interface Operazione {  
    int esegui(int numero);  
}
```

Questa interfaccia dice:

"un'operazione prende un `int` e restituisce un `int`".

Ora possiamo creare un'operazione con una lambda.

```
Operazione doppio = numero -> numero * 2;  
  
System.out.println(doppio.esegui(5));
```

Output:

```
10
```

La lambda:

```
numero -> numero * 2
```

diventa il corpo del metodo `esegui`.

Quando chiami:

```
doppio.esegui(5)
```

Java mette `5` al posto di `numero` e calcola `5 * 2`.

Lambda con due parametri

Una lambda puo ricevere anche piu valori.

In quel caso metti i parametri tra parentesi.

```
(a, b) -> a.compareTo(b)
```

Qui la lambda riceve due valori: `a` e `b`.

Vediamola dentro `sort`, che ordina una lista.

```
public class OrdinaNomi {  
    public static void main(String[] args) {  
        ArrayList nomi = new ArrayList<>();  
        nomi.add("Sara");  
        nomi.add("Luca");  
        nomi.add("Mina");  
  
        nomi.sort((a, b) -> a.compareTo(b));  
  
        System.out.println(nomi);  
    }  
}
```

Output:

```
[Luca, Mina, Sara]
```

`sort` deve sapere come confrontare due elementi.

La lambda:

```
(a, b) -> a.compareTo(b)
```

dice a Java:

"confronta `a` con `b` usando l'ordine naturale delle stringhe".

Quando usarle

Le lambda sono utili quando:

- l'azione è breve
- il codice resta facile da leggere
- stai passando un comportamento a un metodo
- usi collezioni, ordinamenti o stream

Evita lambda troppo lunghe. Se dentro la lambda ci sono molte righe, spesso è meglio creare un metodo con un nome chiaro.

Da ricordare

- Una lambda descrive una piccola azione.
- La freccia `->` significa "usa questo valore per fare questa cosa".
- A sinistra della freccia ci sono i parametri.
- A destra della freccia c'è il codice da eseguire.
- Le lambda sono utili quando un metodo chiede un comportamento da applicare.